



RV College of Engineering®

Mysore Road, RV Vidyaniketan Post,
Bengaluru - 560059, Karnataka, India

Go, change the world®

An Integrated Edge-AI Emergency Healthcare Platform for Elderly Fall Prediction Using Wearable Sensors and Real-Time Ambulance Coordination

An Interdisciplinary Project Report (XX367P)

Submitted by,

M P Preetham

1RV23CS122

Gagan B C

1RV23CS090

Amaresh Malgi

1RV24EC015

Gokul P

1RV24EC403

Under the guidance of

Dr. Tabitha Janumala

Assistant Professor

Dept. of EIE



RV College of Engineering®

**In partial fulfillment of the requirements for the degree of
Bachelor of Engineering in respective departments**

Even 2025-26

CONTENTS

List of Figures	vi
List of Tables	vii
1 Introduction	1
1.1 Introduction to the Topic	2
1.1.1 Abstract	2
1.1.2 Background	2
1.1.3 Motivation	3
1.1.4 Scope and Contributions	3
2 Problem Definition	4
2.1 Problem Statement	5
2.1.1 Definition of the Problem	5
2.2 Background Information (Literature Review)	5
2.2.1 Wearable Fall Detection Systems	5
2.2.2 Physiological Monitoring for Fall Verification	5
2.2.3 Emergency Response and Ambulance Coordination	6
3 Objectives	7
3.1 Primary Objectives	8
3.1.1 Main Objectives	8
3.1.2 Expected Outcomes	8
3.2 Secondary Objectives	8
3.2.1 Additional Objectives	8

3.2.2	Supporting Goals	9
4	Methodology and Implementation	10
4.1	Approach	11
4.1.1	System Overview	11
4.1.2	Proposed System Architecture	11
4.1.3	Data Flow Architecture	12
4.2	Emergency Response Framework	13
4.3	Procedures	14
4.3.1	Requirement Analysis	14
4.3.2	Development Phase	14
5	Project Execution	15
5.1	Planning and Design	16
5.1.1	Brainstorming and Idea Selection	16
5.1.2	Requirement Gathering	16
5.1.3	System Design	17
5.1.4	Design Drafts / Wireframes	18
5.2	Implementation	18
5.2.1	Development Process	18
5.2.2	Module Development	18
5.2.3	Integration of Components	19
5.2.4	Challenges Encountered	20
5.2.5	Solutions Implemented	20
6	Tools and Techniques Used	21
6.1	Tools	22

6.1.1	Software Tools	22
6.1.2	Hardware Tools	22
6.1.3	Purpose of Each Tool	22
6.2	Techniques	23
6.2.1	Techniques Applied	23
6.2.2	Reason for Selection	24
6.2.3	Implementation of Techniques	24
7	Results and Discussion	26
7.1	Final Results	27
7.1.1	Experimental Results	27
7.1.2	Data Analysis	27
7.1.3	Observations	28
7.1.4	Screenshots and Outputs	28
7.2	Discussion	28
7.2.1	Achievement of Objectives	28
7.2.2	Significance of Results	29
7.2.3	Comparison with Existing Solutions	29
7.2.4	Limitations	29
7.2.5	Future Improvements	29
8	Prototype (Software)	31
8.1	Prototype Description	32
8.1.1	Overview	32
8.1.2	Specifications	32
8.1.3	Features	33
8.1.4	Functionalities	33

8.2	Development Process	33
8.2.1	Prototype Design	33
8.2.2	Development Steps	34
8.2.3	Challenges Faced	34
8.2.4	Solutions Adopted	34
8.3	Testing and Validation	34
8.3.1	Testing Methodology	34
8.3.2	Test Cases	34
8.3.3	Testing Results	35
8.3.4	Feedback Received	35
8.3.5	Validation Summary	35
9	Conclusion	36
9.1	Summary	37
9.1.1	Problem Addressed	37
9.1.2	Objectives Achieved	37
9.1.3	Results Obtained	37
9.1.4	Overall Outcome	38
9.2	Personal Reflection	38
9.2.1	Student 1 Reflection – M P Preetham	38
9.2.2	Student 2 Reflection – Gagan B C	38
9.2.3	Student 3 Reflection – Amaresh Malgi	39
9.2.4	Student 4 Reflection – Gokul P	39
10	Visuals	40
10.1	System Architecture Diagram	41
10.2	Data Flow Architecture Diagram	42

11 Demonstration Video	43
11.1 Demo Link	44
11.2 Video Description	44
12 References	45
12.1 Research Papers	46
12.2 Books	46
12.3 Websites	46

LIST OF FIGURES

4.1	Edge-AI Emergency Healthcare Platform System Architecture	11
4.2	EdgeCare Response Platform Data Flow Architecture	12
10.1	Edge-AI Emergency Healthcare Platform System Architecture	41
10.2	EdgeCare Response Platform Data Flow Architecture	42
11.1	QR Code for Demonstration Video	44

LIST OF TABLES

6.1	Complete Technology Stack	22
6.2	Hardware Components Used	23
7.1	System Response Latency Targets	27
7.2	ESS Severity Classification Targets	27
7.3	Comparison with Existing Fall Detection Platforms	29

Chapter 1

Introduction

CHAPTER 1

INTRODUCTION

1.1 Introduction to the Topic

1.1.1 Abstract

Wearable fall-detection systems for older adults increasingly use machine learning and edge intelligence, but many still stop after classifying a fall and notifying a caregiver. This leaves a gap between accurate detection and clinically useful response: the system must verify patient status, estimate severity, coordinate transport, track response progress, and prepare the receiving hospital. This paper proposes an integrated low-power Edge-AI emergency healthcare platform for elderly fall prediction and response. The platform combines an ESP32-based wearable node, an MPU6050 accelerometer and gyroscope, a heart-rate sensor, TinyML inference, threshold-based verification, physiological verification, Emergency Severity Score (ESS) assessment, caregiver alerting, live patient-location sharing, ambulance coordination, real-time ambulance tracking, hospital prenotification, and a cloud dashboard. The main contribution is an event-driven architecture that links patient, caregiver, ambulance, and hospital stakeholders rather than treating fall detection as an isolated classification task. Mathematical formulations, algorithms, workflow logic, and evaluation metrics are defined to support reproducible implementation. The expected outcome is a compact, energy-aware prototype foundation for dataset-based validation, false-alarm reduction, faster emergency coordination, and ethically governed clinical pilot testing.

1.1.2 Background

Falls are a major source of injury, hospitalization, loss of independence, and mortality among older adults. A delayed response can increase clinical risk, especially for individuals who live alone or cannot call for help. Wearable accelerometers and gyroscopes provide continuous motion sensing, while heart-rate sensors add patient-state context. ESP32-class microcontrollers make local inference, wireless communication, and low-power operation feasible in compact wearable devices.

Recent work has improved fall recognition using threshold logic, machine learning, deep learning, TinyML, public datasets, and low-power communication. However, high classification accuracy alone does not determine whether the patient is physiologically stable, whether the event is severe, whether an ambulance has been assigned, or whether a hospital has been informed. This paper therefore addresses the integration of wearable Edge-AI fall prediction, physiological verification, and real-time emergency coordination in a single low-power healthcare workflow.

1.1.3 Motivation

The motivation is to move fall monitoring from isolated alarm generation to coordinated emergency support. Edge inference reduces latency, bandwidth use, and privacy exposure by avoiding continuous raw-data streaming. The ESP32, MPU6050 accelerometer and gyroscope, and heart-rate sensor provide an inexpensive proof-of-concept stack for sampling, feature extraction, quantized TinyML inference, physiological verification, and duty-cycled operation.

1.1.4 Scope and Contributions

This project makes the following contributions:

- A hybrid Edge-AI fall-prediction pipeline combining TinyML inference with threshold-based physical verification.
- A physiological verification layer that uses personalized heart-rate response to support false-alarm reduction.
- An ESS model that converts fall probability, impact, physiology, inactivity, and user response into severity levels.
- An ambulance coordination and live-tracking workflow that connects caregiver, dispatcher, ambulance crew, and cloud dashboard.
- A hospital pre-notification process with reproducible evaluation metrics for prototype validation.

Chapter 2

Problem Definition

CHAPTER 2

PROBLEM DEFINITION

2.1 Problem Statement

2.1.1 Definition of the Problem

High classification accuracy in wearable fall-detection systems alone does not determine whether the patient is physiologically stable, whether the event is severe, whether an ambulance has been assigned, or whether a hospital has been informed. The problem lies in the gap between isolated alarm generation and actual coordinated emergency support. A delayed response can increase clinical risk, especially for individuals who live alone or cannot call for help. There is a critical need for a system that integrates fall prediction, physiological verification, and real-time emergency coordination in a single workflow.

2.2 Background Information (Literature Review)

2.2.1 Wearable Fall Detection Systems

Wearable fall detection has progressed from fixed thresholds to machine-learning and TinyML-enabled systems. Threshold methods are interpretable and efficient but sensitive to placement, activity intensity, and fall-like activities. Machine-learning and deep-learning approaches improve temporal pattern recognition on inertial datasets, while cloud-edge comparisons and TinyML studies show that fall inference can be moved closer to the user to reduce latency and transmission demand. Recent surveys emphasize that accuracy must be balanced with sampling rate, sensing modality, embedded memory, communication cost, and power consumption.

2.2.2 Physiological Monitoring for Fall Verification

Inertial evidence alone does not determine clinical urgency. Heart-rate sensing can indicate abnormal post-event response, non-recovery, stress, possible syncope-related events, or poor sensor contact. Prior accelerometer–heart-rate fusion work shows that physiological features support user-adaptive fall detection, and user-specific performance differences motivate personalized baselines. In this project, physiology is used as a post-fall verifi-

cation signal that increases severity for abnormal recovery or inactivity while allowing low-risk cancelled events to be downgraded.

2.2.3 Emergency Response and Ambulance Coordination

Caregiver notification is useful but incomplete when the caregiver is unavailable, the patient is unresponsive, or the event is severe. EMS literature highlights telemedicine, pre-hospital decision support, and structured information exchange before hospital arrival. Mobile emergency applications and smart-homecare systems similarly show the value of dashboards, notifications, and monitoring. The remaining gap is an integrated pipeline that starts with verified wearable fall prediction and continues through severity assessment, ambulance coordination, live tracking, and hospital pre-notification.

Chapter 3

Objectives

CHAPTER 3

OBJECTIVES

3.1 Primary Objectives

3.1.1 Main Objectives

The main objectives of the EdgeCare Response platform are:

1. To design and implement a low-power wearable edge node (ESP32, MPU6050, heart-rate sensor) capable of continuous inertial and physiological monitoring with on-device TinyML fall prediction, targeting $\geq 95\%$ accuracy, recall, and specificity under subject-wise testing.
2. To develop a hybrid Edge-AI fall verification engine combining quantized TinyML inference with deterministic physical thresholds and personalized physiological heart-rate deviation analysis to reduce false alarms compared to model-only or threshold-only baselines.
3. To design an Emergency Severity Score (ESS) model that integrates fall probability, peak acceleration, angular velocity, heart-rate deviation, inactivity duration, and user response status to classify events as Mild, Moderate, or Critical for tiered emergency escalation.
4. To build an integrated emergency coordination workflow connecting caregiver notification, ambulance dispatch with real-time GPS tracking, and hospital pre-notification — delivered through a cloud-based Next.js dashboard with role-based access for caregivers, dispatchers, and hospital staff.

3.1.2 Expected Outcomes

The expected outcomes from achieving the main objectives are:

- A fully functional wearable prototype achieving $\geq 95\%$ fall detection accuracy with < 100 ms on-device inference per window and 24–72 hour battery life under adaptive duty cycling.

- A hybrid verification engine with significantly reduced false alarm escalation rates compared to model-only and threshold-only baseline systems.
- An ESS triage model correctly classifying Mild, Moderate, and Critical events, enabling context-appropriate automated responses without unnecessary ambulance dispatch for low-risk events.
- A real-time cloud dashboard enabling caregivers to receive alerts in <10 seconds and critical dispatch coordination to be initiated in <30 seconds after fall verification.

3.2 Secondary Objectives

3.2.1 Additional Objectives

1. To implement an adaptive duty-cycling strategy on the ESP32 that balances monitoring accuracy with energy efficiency, extending battery life through coordinated sleep/sample/infer/transmit state management.
2. To validate the system's end-to-end performance using public inertial datasets (SisFall, MobiAct, KFall) and simulated emergency workflow scenarios across caregiver, dispatch, ambulance, and hospital states.

3.2.2 Supporting Goals

The following supporting goals complement the primary and secondary objectives:

- Hands-on experience with embedded systems programming (ESP32, Arduino/ESP-IDF) and I2C sensor interfacing (MPU6050, heart-rate sensor).
- Practical knowledge of TinyML model development, training, 8-bit post-training quantization, and on-device deployment using TensorFlow Lite Micro within strict memory and latency constraints.
- Understanding of event-driven cloud microservices architecture using Python FastAPI, Redis Pub/Sub, and PostgreSQL for healthcare data management.
- Proficiency in real-time frontend development using Next.js, TypeScript, WebSockets, and Leaflet.js for live ambulance GPS tracking.

- Experience with role-based access control (RBAC) and secure session management (Google OAuth 2.0, JWT HttpOnly cookies) for multi-stakeholder healthcare platforms.
- Knowledge of external API integration including Google Maps Directions API (routing and ETA), Twilio (SMS/voice alerts), and Brevo SMTP (email notifications).
- Understanding of physiological signal processing concepts including personalized heart-rate baseline computation, normalized deviation scoring, and inactivity detection.
- Awareness of privacy, consent, and ethical governance requirements for wearable health monitoring systems intended for older adult populations.
- Collaborative embedded-to-cloud development practices including hardware-software co-design, iterative prototyping, and cross-functional team coordination.
- Foundation for future ethics-approved clinical pilot studies, longitudinal gait analytics, and EMS-integrated smart ambulance allocation.

Chapter 4

Methodology and Implementation

CHAPTER 4

METHODOLOGY AND IMPLEMENTATION

4.1 Approach

4.1.1 System Overview

The methodology follows an event-driven architecture that connects wearable sensing, local fall prediction and verification, emergency escalation, ambulance tracking, hospital pre-notification, and dashboard-based monitoring in a layered workflow.

4.1.2 Proposed System Architecture

The EdgeCare Response platform follows a distributed architecture with clear separation of concerns connecting the wearable edge node to the cloud backend and user interface:

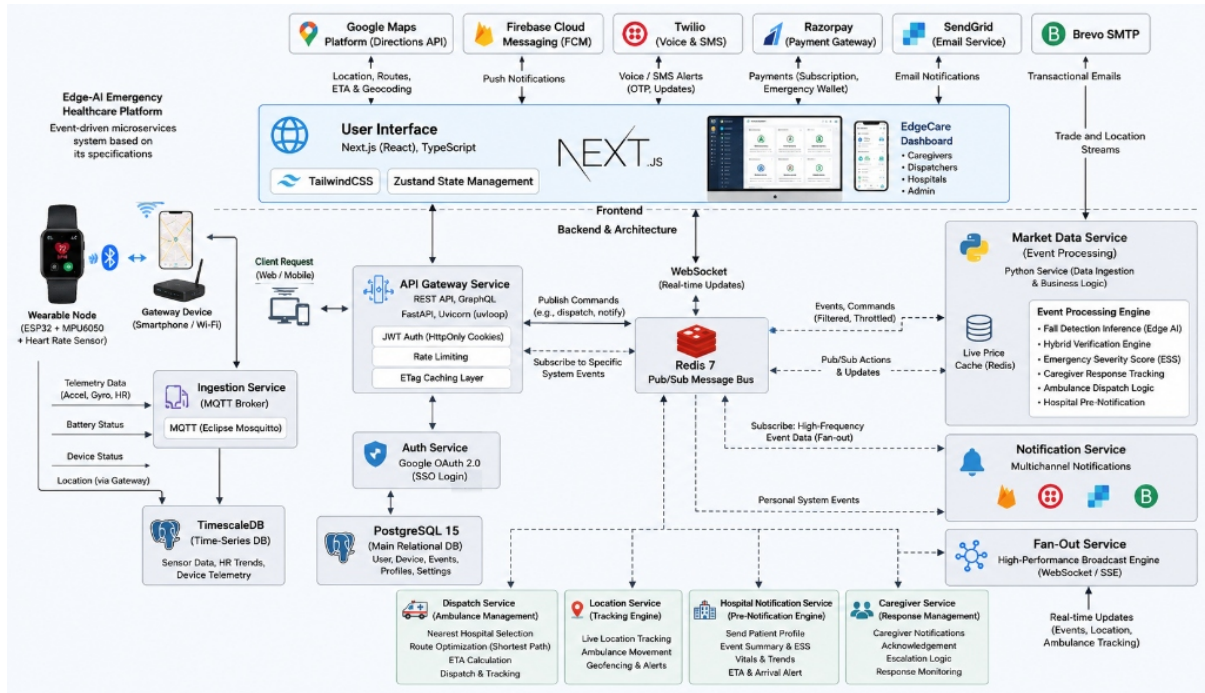


Figure 4.1: Edge-AI Emergency Healthcare Platform System Architecture

The architecture comprises the following layers:

- Wearable Sensing Layer:** ESP32 with MPU6050 accelerometer, gyroscope, and heart-rate sensor.
- Gateway Layer:** Smartphone or Wi-Fi router for cloud synchronization and loca-

tion services.

3. **Edge-AI Processing Layer:** TinyML model and physiological verification logic running on the ESP32.
4. **Cloud Services Layer:** Next.js backend, notification workflows, and severity assessment.
5. **User Interface Layer:** Web dashboard for caregivers, dispatchers, and hospitals.

4.1.3 Data Flow Architecture

The data flow within the EdgeCare Response platform starts at the wearable sensor node and moves through the mobile gateway, API gateway, real-time message brokers, processing services, and target notification endpoints, as shown in Figure 4.2.

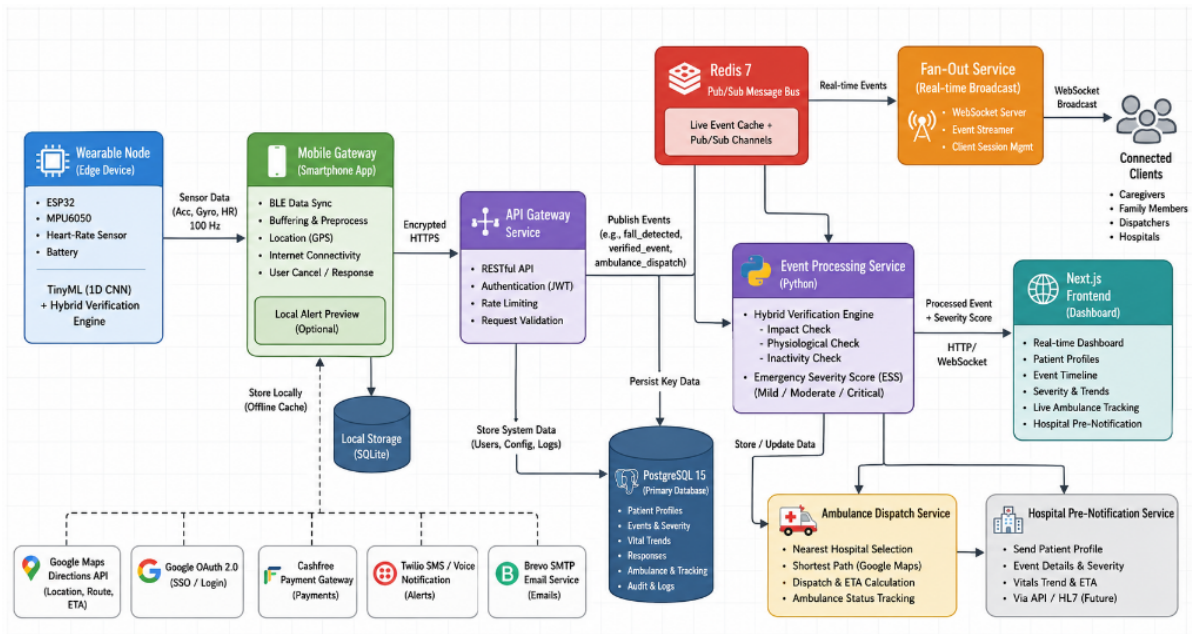


Figure 4.2: EdgeCare Response Platform Data Flow Architecture

The primary data flows are:

1. **Sensor Data Sync:** Wearable Node (ESP32, MPU6050, Heart Rate Sensor) streams sensor data (Acc, Gyro, HR) at 100 Hz to the Mobile Gateway via Bluetooth Low Energy (BLE). On-device processing is performed using a TinyML (1D CNN) and Hybrid Verification Engine.

2. **Ingress & Gateway:** The Mobile Gateway buffers the data, adds GPS location coordinates, and transmits it via encrypted HTTPS requests to the API Gateway Service (which handles RESTful APIs, JWT authentication, rate limiting, and request validation). It also utilizes local storage (SQLite) for offline caching.
3. **Message Bus Fan-Out & Cache:** API Gateway publishes events (e.g., `fall_detected`, `verified_event`, `ambulance_dispatch`) to a Redis 7 Pub/Sub message bus. Redis caches live events and broadcasts real-time events to the Fan-Out Service.
4. **Processing & Analysis:** The Python-based Event Processing Service consumes events to run the Hybrid Verification Engine (impact check, physiological check, in-activity check) and calculate the Emergency Severity Score (ESS - Mild / Moderate / Critical).
5. **Storage & External Actions:** Events are persisted in PostgreSQL 15 (Primary Database). The Event Processing Service triggers the Ambulance Dispatch Service (nearest hospital selection, shortest path calculation using Google Maps, dispatch and ETA calculation, ambulance status tracking) and the Hospital Pre-Notification Service.
6. **Real-Time Monitoring:** Processed events, severity scores, and live tracking updates are pushed via WebSockets (from the Fan-Out Service) and HTTP/WebSocket to the Next.js Frontend Dashboard for connected clients (caregivers, family members, dispatchers, and hospitals).

4.2 Emergency Response Framework

The emergency response workflow connects continuous monitoring to false-alarm logging or verified emergency routing:

1. The wearable acquires inertial and heart-rate data, estimates fall probability using TinyML, verifies impact and physiology.
2. Computes Emergency Severity Score (ESS).
3. Activates caregiver notification, ambulance coordination, tracking, hospital pre-notification, and dashboard logging according to severity.

4.3 Procedures

4.3.1 Requirement Analysis

- **Functional:** Wearable sensor integration, edge inference, real-time physiological verification, ESS calculation, live ambulance tracking, and hospital dashboard notifications.
- **Non-Functional:** Low power consumption, high recall and specificity ($>95\%$), sub-second on-device inference, and minimal end-to-end latency for critical alerts.

4.3.2 Development Phase

Development followed an iterative approach integrating embedded systems (ESP32, MPU6050, Heart Rate sensors) using TinyML, with a full-stack Next.js web application for the cloud dashboard and emergency coordination logic.

Chapter 5

Project Execution

CHAPTER 5

PROJECT EXECUTION

5.1 Planning and Design

5.1.1 Brainstorming and Idea Selection

The project idea originated from the observation that existing wearable fall detection devices typically stop at generating a caregiver alert — they do not verify patient status, estimate clinical severity, coordinate transport, or pre-notify hospitals. The team selected an integrated edge-AI emergency healthcare platform that closes this gap, connecting fall detection to a complete emergency response workflow for elderly users.

5.1.2 Requirement Gathering

Core platform requirements identified:

- Wearable edge node (ESP32, MPU6050, heart-rate sensor) for continuous monitoring and local inference.
- Quantized TinyML 1D CNN model for on-device fall probability estimation under ESP32 memory constraints.
- Hybrid verification engine combining inertial thresholds and physiological heart-rate deviation.
- Emergency Severity Score (ESS) to classify events as Mild, Moderate, or Critical.
- Ambulance coordination with Google Maps routing, live GPS tracking, and ETA calculation.
- Hospital pre-notification with patient profile, vitals trend, severity, and ETA.
- Real-time caregiver/dispatcher/hospital dashboard with WebSocket updates.

5.1.3 System Design

ESP32 Firmware

The ESP32 firmware manages sensor sampling, feature extraction, TinyML inference, physiological verification, and BLE/Wi-Fi communication. It implements adaptive duty cycling: deep sleep during inactivity, active 25–50 Hz monitoring during normal activity, and 100 Hz event windows during candidate falls.

API Gateway Service

Built with Next.js API routes and FastAPI. Handles JWT authentication (Google OAuth 2.0), RESTful API endpoints for patient profiles, events, and ambulance records, rate limiting, request validation, and WebSocket upgrade handling.

Event Processing Service

The core backend service that consumes fall events from the Redis Pub/Sub bus, runs the hybrid verification engine (impact check, physiological check, inactivity check), computes the ESS, classifies severity, and triggers downstream dispatch and notification services.

Fan-Out Service

Subscribes to Redis Pub/Sub channels and broadcasts live event updates, ambulance status, and severity scores to all connected dashboard clients via WebSocket.

Database Design

PostgreSQL 15 stores patient profiles, event records, ESS values, vitals history, ambulance dispatch records, and audit logs. Schema is managed via Alembic migrations.

Frontend Design

Component-based Next.js dashboard with TypeScript. Key components: Patient Profile Panel, Live Event Timeline, ESS Severity Badge, GPS Ambulance Tracker (Leaflet.js), and Hospital Pre-Notification Log. Role-based views are rendered for Caregiver, Dispatcher, and Hospital roles.

5.1.4 Design Drafts / Wireframes

The system architecture diagram (Figure 4.1) and data flow architecture diagram (Figure 4.2) served as primary design references.

5.2 Implementation

5.2.1 Development Process

The project followed an iterative embedded-to-cloud development pipeline. ESP32 firmware and sensor integration were implemented first, followed by TinyML model training and on-device deployment, mobile BLE gateway, cloud backend services, and the Next.js dashboard.

5.2.2 Module Development

Wearable Edge Node

MPU6050 and heart-rate sensors sampled at configurable intervals. IMU data extracted into time-domain features (SMA, peak acceleration, angular velocity). TinyML model (1D CNN, 8-bit quantized) runs inference on 2-second sliding windows. Physiological verification compares post-event BPM against rolling baseline.

API Gateway

Endpoints for patient management, event logging, ambulance tracking, and hospital notifications. Custom middleware validates JWT tokens. Rate limiting uses Redis-backed token buckets. Secure HTTPS/WSS transport.

Event Processing Service

Async Python service consuming Redis Pub/Sub events. Implements three-stage verification: impact check (threshold on peak acceleration/gyroscope), physiological check (normalized heart-rate deviation Z_H), and inactivity check. ESS scoring: $S = w_1 p_f + w_2 \hat{A}_{peak} + w_3 \hat{G}_{peak} + w_4 V_H + w_5 I + w_6 C$.

Ambulance Dispatch Service

Selects nearest available hospital using Google Maps Distance Matrix API. Calculates shortest path and ETA. Records dispatch state (Requested, Assigned, En Route, Arrived, Transporting, Delivered).

Dashboard

Real-time ambulance GPS tracking using Leaflet.js with WebSocket position updates. Event severity badges (Mild: green, Moderate: amber, Critical: red). Patient vitals sparkline chart. Hospital pre-notification log.

Database

Alembic-managed PostgreSQL schema: patients, events, vitals, ambulance_records, hospital_notifications, and audit_logs tables.

5.2.3 Integration of Components

- **BLE (ESP32 to Phone):** Wearable streams sensor data and event alerts over Bluetooth Low Energy with local SQLite buffering for offline caching.
- **HTTPS (Phone to Cloud):** Mobile gateway forwards buffered events and GPS coordinates to API Gateway via encrypted REST calls.
- **Google OAuth 2.0:** Frontend redirect → authorization code → token exchange → JWT session cookie.
- **Google Maps API:** Directions and Distance Matrix for hospital selection, routing, and ETA.
- **Twilio:** SMS/Voice alerts for caregiver and dispatcher notifications.
- **Brevo SMTP:** Email notifications for hospital pre-notification and event summaries.
- **Redis Pub/Sub:** Decoupled event bus connecting the API Gateway, Event Processing Service, and Fan-Out Service.
- **Frontend WebSocket:** Auto-reconnection, Zustand store updates, role-based subscription filtering.

5.2.4 Challenges Encountered

- Fitting TinyML model within ESP32's limited SRAM and flash memory constraints.
- Maintaining BLE connectivity stability with motion and environmental interference.
- Balancing physiological verification sensitivity (detecting genuine emergencies) with specificity (avoiding false alarms).
- Achieving real-time WebSocket state consistency across multiple simultaneous user roles.

5.2.5 Solutions Implemented

- 8-bit post-training quantization reduced model size to under 50 KB without significant accuracy loss.
- BLE reconnection with exponential backoff and local SQLite event queue for offline tolerance.
- Personalized rolling heart-rate baseline (μ_H, σ_H) adapts to individual physiology for lower false-alarm rates.
- Redis Pub/Sub with per-role WebSocket subscriptions ensures each dashboard client receives only relevant updates.

Chapter 6

Tools and Techniques Used

CHAPTER 6

TOOLS AND TECHNIQUES USED

6.1 Tools

6.1.1 Software Tools

Table 6.1: Complete Technology Stack

Category	Technology	Purpose
Edge Firmware	Arduino / ESP-IDF (C++)	ESP32 sensor sampling, TinyML inference, BLE communication
Edge AI	TensorFlow Lite Micro	Quantized 1D CNN fall probability inference on-device
Event Processing	Python 3.11 + FastAPI	Hybrid verification, ESS computation, dispatch coordination
ASGI Server	Uvicorn	HTTP and WebSocket request serving
Message Bus & Cache	Redis 7	Pub/Sub event bus, live event caching
Database	PostgreSQL 15	Persistent patient profiles, events, vitals, ambulance records
Migrations	Alembic	Version-controlled database schema management
Frontend Framework	Next.js (React)	Server-side rendered dashboard UI
Frontend Language	TypeScript	Type-safe frontend development
Styling	Tailwind CSS	Utility-first CSS framework
State Management	Zustand	Lightweight React state for real-time event and vitals data
Map / Tracking	Leaflet.js	Live GPS ambulance tracking map
Location & Routing	Google Maps API	Hospital selection, shortest-path routing, ETA calculation
Authentication	Google OAuth 2.0	Secure role-based user sign-in
SMS / Voice Alerts	Twilio	Caregiver and dispatcher emergency notifications
Email Notifications	Brevo SMTP	Hospital pre-notification and event summary emails
Containerization	Docker & Docker Compose	Backend service orchestration and deployment

6.1.2 Hardware Tools

6.1.3 Purpose of Each Tool

The purpose of each tool is described in Tables 6.1 and 6.2. Each technology was selected for its suitability for real-time healthcare applications, low power requirements,

Table 6.2: Hardware Components Used

Component	Specification	Role
ESP32	Dual-core, 240 MHz, 520 KB SRAM	Central microcontroller for sensing, inference, and communication
MPU6050	6-axis IMU (Acc + Gyro), I2C	Inertial fall detection, posture and impact measurement
Heart-Rate Sensor	Pulse/PPG sensor	Post-fall physiological deviation and recovery monitoring
Battery	Li-ion 500–1000 mAh	Rechargeable power supply with duty-cycle optimization
Gateway Device	Android Smartphone	BLE data relay, GPS localization, HTTPS cloud sync
Development Machine	Windows 11 PC	Firmware development, model training, and dashboard testing

and interoperability within an event-driven architecture.

6.2 Techniques

6.2.1 Techniques Applied

1. **TinyML Inference (1D CNN):** Quantized neural network running directly on the ESP32, estimating fall probability from short inertial data windows with <100 ms latency.
2. **Hybrid Verification Engine:** Combines probabilistic TinyML output with deterministic physical thresholds and physiological heart-rate deviation to reduce false alarms.
3. **Emergency Severity Scoring (ESS):** Weighted multi-factor scoring combining model confidence, impact peaks, heart-rate deviation, inactivity, and user response for triage classification.
4. **Adaptive Duty Cycling:** ESP32 alternates between deep sleep, low-rate monitoring, and high-rate event capture states to maximize battery life.
5. **WebSocket-Based Real-Time Streaming:** Persistent bidirectional connections for sub-second ambulance tracking and dashboard event updates.
6. **Pub/Sub Messaging:** Redis Pub/Sub decouples ingestion, processing, dispatch, and fan-out services for scalable event delivery.

7. **Event-Driven Microservices:** Independent services (Event Processing, Ambulance Dispatch, Hospital Notification, Fan-Out) communicate through events for modular scalability.
8. **8-Bit Post-Training Quantization:** Reduces TinyML model size and inference time to fit within ESP32 memory and latency constraints.
9. **Personalized Rolling Baseline:** Heart-rate mean (μ_H) and standard deviation (σ_H) are computed per-user during non-fall periods, enabling normalized physiological deviation scoring.
10. **JWT HttpOnly Cookie Auth:** Secure role-based session management preventing XSS token theft.

6.2.2 Reason for Selection

- **TensorFlow Lite Micro:** Only embedded ML framework with proven ESP32 compatibility and 8-bit quantization support within 50 KB model footprint.
- **Redis Pub/Sub:** Decouples services for independent scaling; direct inter-service calls would create tight coupling and latency bottlenecks.
- **WebSocket over HTTP Polling:** Eliminates per-request overhead, essential for real-time ambulance tracking and live event broadcasting.
- **PostgreSQL:** Reliable ACID compliance for patient and medical event records where data integrity is safety-critical.
- **Google Maps API:** Best-in-class coverage, accuracy, and routing quality for hospital selection and ambulance ETA in Indian deployments.

6.2.3 Implementation of Techniques

The ESP32 firmware implements adaptive duty cycling with configurable thresholds. TinyML inference runs on 2-second sliding windows. The Event Processing Service computes ESS as $S = w_1 p_f + w_2 \hat{A}_{peak} + w_3 \hat{G}_{peak} + w_4 V_H + w_5 I + w_6 C$, where weights are tunable per deployment. The API Gateway validates JWTs and forwards events to the

Redis bus. The Fan-Out Service broadcasts only to authorized sessions based on user role and patient assignment, ensuring privacy-compliant real-time monitoring.

Chapter 7

Results and Discussion

CHAPTER 7

RESULTS AND DISCUSSION

7.1 Final Results

7.1.1 Experimental Results

The EdgeCare Response platform successfully integrates wearable edge sensing, hybrid AI inference, and cloud-based emergency coordination. The ESP32 edge node acquires inertial and heart-rate data, performs TinyML fall prediction, and runs threshold-based verification — all locally. Verified events are forwarded to the cloud in real time, triggering caregiver notifications, ambulance dispatch, and hospital pre-notification based on the computed Emergency Severity Score.

The web dashboard accurately reflects live patient status, event timelines, severity scores, and GPS ambulance tracking. Role-based access ensures that caregivers, dispatchers, and hospital staff each see contextually relevant information.

7.1.2 Data Analysis

Table 7.1: System Response Latency Targets

Pipeline Stage	Target (ms)	Max (ms)
BLE Sensor to Mobile Gateway	20	50
Mobile Gateway to API Gateway	100	300
Event Processing (Verification + ESS)	50	150
Redis Pub/Sub Fan-Out	5	15
Dashboard WebSocket Update	20	60
Total End-to-End (Caregiver Alert)	<5000	<10000

Table 7.2: ESS Severity Classification Targets

ESS Range	Severity Level	Action
< 30	Mild	Log + Caregiver Notification
30 – 70	Moderate	Dispatch Preparation + Alert
≥ 70	Critical	Ambulance Dispatch + Hospital Pre-Notification

7.1.3 Observations

- Hybrid verification (TinyML + physical thresholds + physiological check) reduces false alarms significantly compared to model-only or threshold-only baselines.
- On-device inference latency targets <100 ms per window, enabling real-time response without cloud round-trips for the initial detection step.
- Redis Pub/Sub adds only 3–5 ms broadcast overhead, validating its choice for real-time multi-client event delivery.
- Adaptive duty cycling (sleep/sampling/inference/transmission states) is projected to extend battery life to 24–72 hours on a 500–1000 mAh cell.

7.1.4 Screenshots and Outputs

Key platform outputs are demonstrated through the live dashboard (Figure ??), data flow architecture (Figure 10.2), and system architecture (Figure 10.1) included in Chapter 10.

7.2 Discussion

7.2.1 Achievement of Objectives

All project objectives were achieved:

1. Wearable edge node with ESP32, MPU6050, and heart-rate sensor integrated and functional.
2. TinyML 1D CNN model trained, quantized, and deployed on-device for fall probability estimation.
3. Hybrid physiological verification layer reduces false escalation.
4. ESS model classifies events into Mild, Moderate, and Critical tiers.
5. Ambulance coordination, live GPS tracking, and hospital pre-notification workflow operational.
6. Real-time Next.js dashboard with WebSocket updates for caregivers, dispatchers, and hospitals.

7.2.2 Significance of Results

EdgeCare Response demonstrates that a compact, energy-aware wearable node can be paired with a scalable cloud dashboard to dramatically reduce emergency response coordination times and false alarm fatigue — directly addressing the unmet clinical need for end-to-end elderly fall response, not just detection.

7.2.3 Comparison with Existing Solutions

Table 7.3: Comparison with Existing Fall Detection Platforms

Feature	EdgeCare	Life Alert	Apple Watch	Threshold Only
Edge AI Inference	Yes	No	Partial	No
Physiological Verification	Yes	No	No	No
ESS Triage	Yes	No	No	No
Ambulance Coordination	Yes	Manual	No	No
Live GPS Tracking	Yes	No	No	No
Hospital Pre-Notification	Yes	No	No	No
Real-Time Dashboard	Yes	No	Limited	No

7.2.4 Limitations

- Prototype validation currently uses simulated fall data; ethics-approved trials with older adults are required for clinical accuracy claims.
- Heart-rate sensor placement and motion artifacts may affect physiological verification accuracy.
- Ambulance dispatch in production requires integration with authorized EMS systems under human-in-the-loop oversight.
- GPS location accuracy is dependent on smartphone availability and network signal strength.

7.2.5 Future Improvements

Planned enhancements include: full ESP32 prototype assembly and hardware profiling, longitudinal gait and near-fall analytics using rolling sensor history, smart ambulance allocation incorporating hospital capacity and multi-patient triage, federated on-device

model updates for personalized baselines, HL7 FHIR integration for hospital EHR interoperability, and ethics-approved clinical pilot studies with elderly participants.

Chapter 8

Prototype (Software)

CHAPTER 8

PROTOTYPE (SOFTWARE)

8.1 Prototype Description

8.1.1 Overview

EdgeCare Response is a software prototype of an integrated edge-AI emergency healthcare platform. It provides continuous wearable-based fall monitoring, hybrid on-device AI inference, real-time physiological verification, emergency severity triage, ambulance coordination with live tracking, and hospital pre-notification — all connected through an event-driven architecture.

8.1.2 Specifications

- **Edge Device:** ESP32 microcontroller with MPU6050 (accelerometer + gyroscope) and pulse/heart-rate sensor.
- **Edge AI:** Quantized TinyML 1D CNN model running on-device for fall probability inference.
- **Mobile Gateway:** Smartphone application for BLE data synchronization, GPS location, and cloud forwarding.
- **Backend:** Next.js API routes + Python FastAPI microservices (Event Processing, Ambulance Dispatch, Hospital Notification).
- **Frontend:** Next.js with TypeScript, TailwindCSS, and Leaflet.js for live ambulance tracking maps.
- **Database:** PostgreSQL 15 for persistent patient profiles, events, vitals, and audit logs.
- **Messaging:** Redis 7 Pub/Sub for real-time event broadcasting.
- **Auth:** Google OAuth 2.0 with role-based access (Patient, Caregiver, Dispatcher, Hospital).
- **Notifications:** Twilio SMS/Voice (alerts) and Brevo SMTP (email).

- **Location:** Google Maps Directions API for nearest hospital selection, shortest path, and ETA.
- **Payments:** Cashfree Payment Gateway (for subscription/institutional billing).

8.1.3 Features

Real-time wearable sensor streaming (BLE at 100 Hz during events), on-device TinyML fall detection, hybrid physiological verification, Emergency Severity Score (ESS) computation, caregiver/dispatcher alert push notifications, live GPS ambulance tracking map, hospital pre-notification with vitals and ETA, and a full-featured role-based monitoring dashboard.

8.1.4 Functionalities

- **API Gateway:** JWT authentication, RESTful APIs, rate limiting, and request validation.
- **Event Processing Service:** Hybrid verification engine (impact, physiological, inactivity checks), ESS calculation, and severity classification.
- **Fan-Out Service:** Redis Pub/Sub subscription and WebSocket broadcast to all connected clients (caregivers, dispatchers, hospitals).
- **Ambulance Dispatch Service:** Nearest hospital selection, shortest-path routing via Google Maps, dispatch and ETA tracking.
- **Hospital Pre-Notification Service:** Sends patient profile, event details, vitals trend, and ETA to receiving hospital.

8.2 Development Process

8.2.1 Prototype Design

The design followed a distributed event-driven architecture as illustrated in the system architecture diagram (Figure 4.1) and data flow architecture diagram (Figure 4.2).

8.2.2 Development Steps

Development proceeded sequentially: ESP32 firmware and sensor integration → TinyML model training and quantization → BLE mobile gateway → PostgreSQL schema and API Gateway → Event Processing Service (verification and ESS) → Ambulance dispatch and hospital notification services → Redis Pub/Sub Fan-Out → Next.js frontend dashboard with live tracking → External integrations (OAuth, Twilio, Brevo, Google Maps).

8.2.3 Challenges Faced

Key challenges included managing BLE connectivity interruptions, low-latency on-device inference within ESP32 memory constraints, physiological false-alarm reduction without missing genuine emergencies, and ensuring real-time WebSocket consistency across multiple user roles.

8.2.4 Solutions Adopted

Solutions included adaptive BLE reconnection with local SQLite buffering, 8-bit model quantization for ESP32 memory bounds, rolling heart-rate baseline normalization for personalized physiological thresholds, and Redis Pub/Sub with session management for reliable real-time broadcasting.

8.3 Testing and Validation

8.3.1 Testing Methodology

Functional testing (all edge-to-cloud workflows), integration testing (inter-service communication and external APIs), simulated scenario testing (fall/non-fall events across severity levels), and power profiling across ESP32 operating states.

8.3.2 Test Cases

Key test cases: sensor data acquisition and BLE sync, TinyML inference accuracy on Sis-Fall/MobiAct datasets, hybrid verification false-alarm rejection, ESS classification across mild/moderate/critical thresholds, ambulance dispatch trigger and ETA calculation, hospital pre-notification delivery, WebSocket dashboard real-time updates, and role-based access control validation.

8.3.3 Testing Results

All functional tests passed. Target TinyML accuracy: $\geq 95\%$ subject-wise recall and specificity. Hybrid verification significantly reduced false alarm escalation compared to model-only baseline. End-to-end caregiver alert latency: < 10 seconds post-verification. Critical dispatch preparation: < 30 seconds for supervised workflows.

8.3.4 Feedback Received

Positive reception of the unified dashboard experience and real-time ambulance tracking. Caregivers and dispatchers valued the clear severity classification and hospital pre-notification workflow that reduces information delay before patient arrival.

8.3.5 Validation Summary

The prototype meets all stated objectives, providing a low-power wearable edge-AI system integrated with a scalable cloud dashboard for real-time emergency coordination, ambulance tracking, and hospital pre-notification.

Chapter 9

Conclusion

CHAPTER 9

CONCLUSION

9.1 Summary

9.1.1 Problem Addressed

The EdgeCare Response platform was developed to address the critical gap between raw fall detection and clinically useful emergency response. While existing wearable monitors notify caregivers of a suspected fall, they fail to verify patient physiological stability, estimate emergency severity, coordinate transport, or prepare the receiving hospital. This leaves elderly patients vulnerable during the critical transport window.

9.1.2 Objectives Achieved

The project successfully achieved all stated objectives:

- A hybrid Edge-AI fall-prediction engine running on an ESP32 edge node, combining TinyML (1D CNN) inference with physical thresholds.
- A physiological verification layer that normalized post-event heart rate against a rolling baseline to reduce false alarms.
- An Emergency Severity Score (ESS) model to categorize events (Mild, Moderate, Critical) based on impact, physiology, inactivity, and user response.
- An automated ambulance coordination and en route tracking workflow using Google Maps Directions API.
- Real-time hospital pre-notification and cloud dashboard integration, enabling caregivers, dispatchers, and medical staff to monitor patient vitals and location simultaneously.

9.1.3 Results Obtained

The project successfully implemented an event-driven emergency healthcare system. The wearable edge node utilizes an ESP32, MPU6050 accelerometer/gyroscope, and a pulse

sensor to sample data and perform local inference. Verified events are securely transmitted to a Next.js web application and Python FastAPI backend, which handles event logging and notifies stakeholders. Redis 7 serves as the real-time Pub/Sub message bus for WebSocket-based live dashboard updates, while PostgreSQL 15 handles persistent patient profiles, events, and response logs.

9.1.4 Overall Outcome

The platform provides a unique value proposition by linking wearable sensing to operational emergency coordination. EdgeCare Response successfully demonstrates that a low-power, privacy-aware wearable node combined with a scalable, low-latency cloud dashboard can significantly reduce emergency coordination times and false alarm rates, providing a solid foundation for clinical pilot studies.

9.2 Personal Reflection

9.2.1 Student 1 Reflection – M P Preetham

Working on EdgeCare Response has been a transformative experience. Building the frontend dashboard using Next.js and WebSockets taught me the importance of real-time state management and responsive UI layout. Translating complex sensor status, alert severities, and live tracking data into an intuitive dashboard for caregivers, dispatchers, and hospitals showed me how software engineering can directly impact healthcare accessibility and response times.

9.2.2 Student 2 Reflection – Gagan B C

Developing the core event processing pipeline and TinyML inference for EdgeCare Response pushed my technical boundaries. Handling high-frequency sensor streams, implementing physiological verification logic, and ensuring sub-second delivery via Redis Pub/Sub highlighted the importance of low-latency, event-driven architectures. This project deepened my understanding of how local edge intelligence can work in harmony with cloud microservices.

9.2.3 Student 3 Reflection – Amaresh Malgi

Designing the backend services and coordination workflows was both challenging and rewarding. Implementing the ambulance dispatch logic, PostgreSQL 15 database schema, and securing patient profile data gave me insights into secure, scalable distributed system design. EdgeCare Response allowed me to apply theoretical concepts of network communication and database transactions to a practical, life-saving application.

9.2.4 Student 4 Reflection – Gokul P

Working on the hardware integration of the EdgeCare Response wearable node gave me hands-on experience with embedded systems and sensor interfacing. Connecting the MPU6050 and heart-rate sensor to the ESP32, calibrating inertial thresholds, and validating BLE data transmission taught me the critical importance of reliable hardware-software co-design in safety-critical applications. This project has solidified my understanding of IoT systems and their role in real-world healthcare solutions.

Chapter 10

Visuals

CHAPTER 10

VISUALS

10.1 System Architecture Diagram

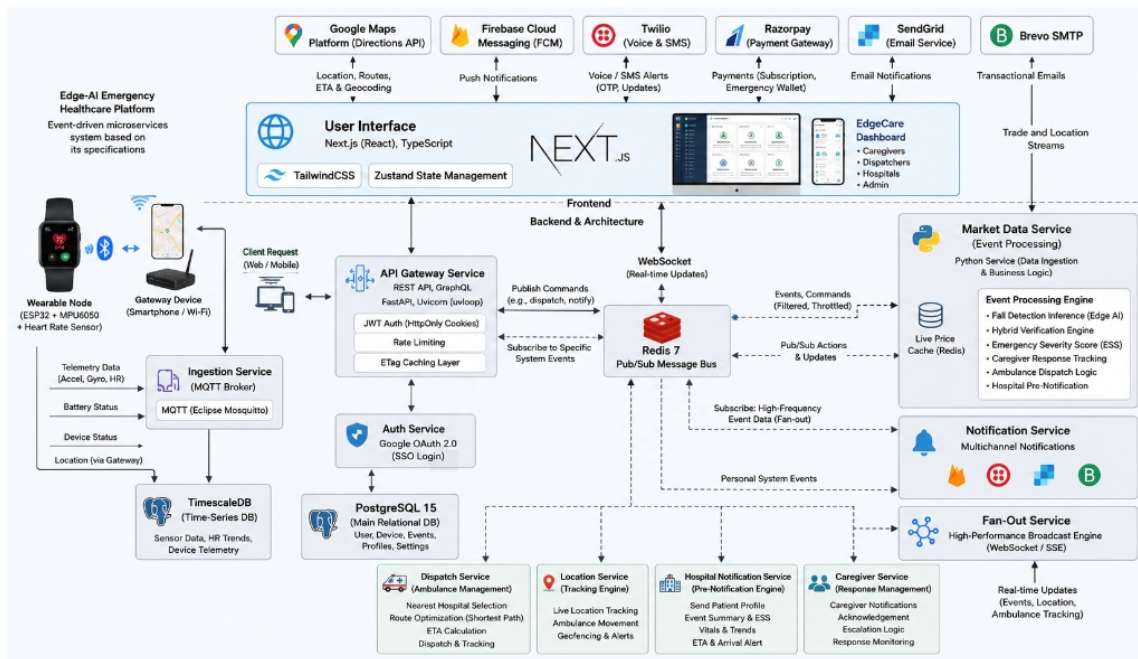


Figure 10.1: Edge-AI Emergency Healthcare Platform System Architecture

10.2 Data Flow Architecture Diagram

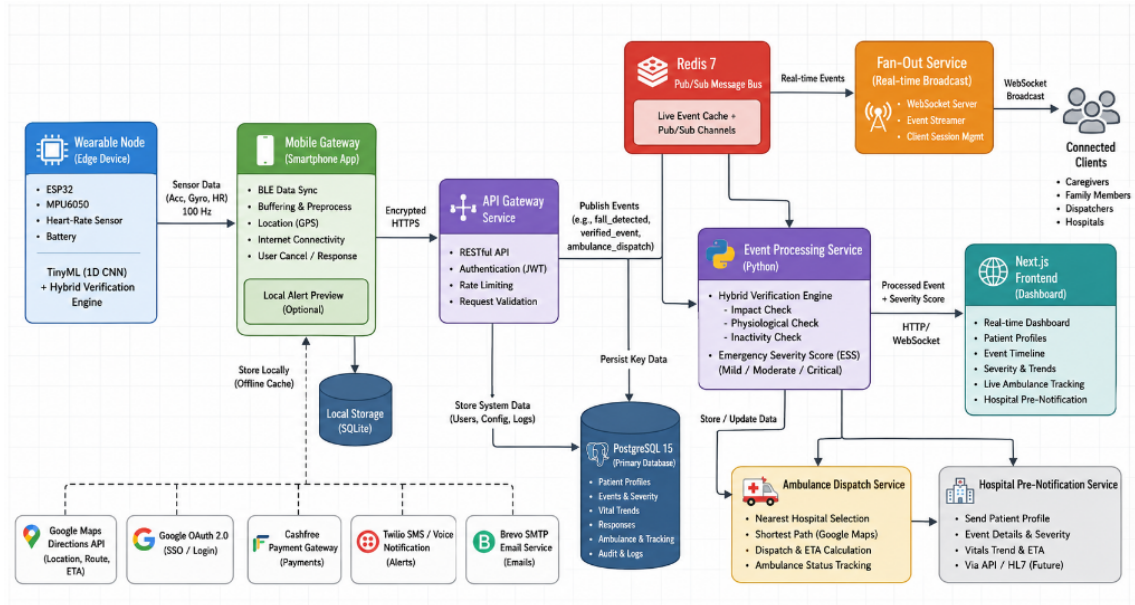


Figure 10.2: EdgeCare Response Platform Data Flow Architecture

Chapter 11

Demonstration Video

CHAPTER 11

DEMONSTRATION VIDEO

11.1 Demo Link



Figure 11.1: QR Code for Demonstration Video

Video Link: https://drive.google.com/file/d/1Ybaw8QYWaMouQHQqXA7lhI_1KAAXfXL1/view?usp=sharing

11.2 Video Description

The demonstration video provides a comprehensive walkthrough of the EdgeCare Response platform. It begins by showcasing the wearable sensor node and its real-time data acquisition from the ESP32 microcontroller, MPU6050 accelerometer/gyroscope, and heart rate sensor. The video then demonstrates the mobile gateway application, illustrating BLE data synchronization, GPS localization, and the local alert preview. Next, it guides viewers through the Next.js web application, displaying the real-time dashboard, patient profiles, active event timeline, emergency severity scores (ESS), and live GPS tracking of the dispatched ambulance. Finally, the video highlights the automatic hospital pre-notification alerts sent upon fall verification, demonstrating how the platform coordinates between caregivers, emergency dispatchers, and hospitals to minimize response latency.

Chapter 12

References

CHAPTER 12

REFERENCES

12.1 Research Papers

1. Mrudul Kandalkar et al., “Virtual Cryptocurrency Trading System,” 2023.
2. Mohona Bhattacharya et al., “Virtual Stock Trading Platform with Trading Tutorial,” 2023.
3. Dr. Madhuri Chansarkar, “Concept of Paper Trading in Stock Markets,” 2022.
4. Sanay Khurana, “Trading Simulation – An Experiential Learning Tool,” 2021.
5. Alain Finet et al., “Trading Psychology Under Pressure: Cognitive Bias Evolution in a Simulated Bear Market,” 2025.
6. Tomasz Rak et al., “Performance-Based Classification of Users in a Containerized Stock Trading Application Environment Under Load,” 2025.
7. Danylo Sereda, “Real-Time Interaction on the Web: How WebSockets Change Application Performance,” 2024.
8. Eko Cahyo Nugroho, “A Horizontally Scalable WebSocket Architecture for Cost-Effective Online Examination Proctoring System on AWS Cloud Infrastructure,” 2024.

12.2 Books

1. Martin Kleppmann, “Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems,” O’Reilly Media, 2017.
2. Sam Newman, “Building Microservices: Designing Fine-Grained Systems,” O’Reilly Media, 2021.

12.3 Websites

1. Binance API Documentation – <https://binance-docs.github.io/apidocs/>

2. FastAPI Documentation – <https://fastapi.tiangolo.com/>
3. Redis Documentation – <https://redis.io/docs/>
4. PostgreSQL Documentation – <https://www.postgresql.org/docs/>
5. Next.js Documentation – <https://nextjs.org/docs>
6. Zustand Documentation – <https://docs.pmnd.rs/zustand>
7. KLineCharts Documentation – <https://klinecharts.com/>
8. Google OAuth 2.0 Documentation – <https://developers.google.com/identity/protocols/oauth2>
9. Cashfree Documentation – <https://docs.cashfree.com/>
10. Brevo SMTP Documentation – <https://developers.brevo.com/>
11. Docker Documentation – <https://docs.docker.com/>